



# Comment fonctionne un émulateur Super Nintendo

Guide pédagogique complet — du transistor au pixel, du processeur au son

---

Émulateur écrit **from scratch en Rust** — CPU 65C816, PPU graphique, APU sonore (SPC700 + S-DSP), DMA/HDMA, cartouches LoROM/HiROM et **quatre coprocesseurs de cartouche** (SuperFX, SA-1, DSP-1, CX4), plus l'application qui l'entoure.

Document technique et pédagogique · Version 3.0 — édition détaillée

# Table des matières

1. Qu'est-ce qu'émuler une console ?
2. Petit rappel : bits, octets et hexadécimal
3. Vue d'ensemble : qui parle à qui
4. La cartouche et la carte mémoire
5. Le cœur : le processeur 65C816
6. Le chef d'orchestre : le timing et l'ordonnanceur
7. La puce graphique : le PPU
8. Le déménageur : DMA et HDMA
9. Le son : l'APU (SPC700 + S-DSP)
10. Les puces de cartouche : quatre processeurs de plus
11. Pourquoi « au cycle près » : l'affaire Terranigma
12. La boucle d'émulation : tout synchroniser
13. Au-delà de la console : sauvegardes et save states
14. L'application autour de l'émulateur
15. Deux essais qui n'ont pas été gardés
16. Comment cet émulateur a été construit et vérifié
17. Glossaire

## 1. Qu'est-ce qu'émuler une console ?

Une console de jeu comme la Super Nintendo (SNES / Super Famicom, sortie en 1990) est un ensemble de **puces électroniques** qui coopèrent : un processeur, une puce vidéo, une puce audio, de la mémoire. **Émuler** la console, c'est écrire un programme qui **imite le comportement de chacune de ces puces**, assez fidèlement pour que le code d'origine d'un jeu — les mêmes octets que sur la cartouche — s'exécute et produise exactement la même image et le même son que sur la vraie machine.

### 💡 Analogie

Émuler, ce n'est pas « refaire le jeu ». C'est construire une **fausse console** si convaincante que la vraie cartouche ne voit pas la différence. Le jeu croit parler à une vraie puce vidéo ; en réalité il parle à quelques milliers de lignes de Rust qui répondent exactement comme le silicium d'origine l'aurait fait.

### Pourquoi la fidélité est difficile

Le défi n'est pas d'imiter une puce isolée : c'est de faire **cohabiter toutes les puces au rythme exact**. Les programmeurs de jeux des années 90 exploitaient le moindre détail de synchronisation — par exemple changer une couleur pile au moment où le canon à électrons du téléviseur passe à la ligne suivante, pour obtenir un dégradé. Si l'émulateur se trompe de quelques cycles, l'effet casse. Un bon émulateur doit donc reproduire non seulement *ce que* font les puces, mais *quand* elles le font.

La SNES est une machine **16 bits**. Ses composants principaux :

- **Le CPU 65C816** — le cerveau : il exécute la logique du jeu (physique, IA, score...).
- **Le PPU** (*Picture Processing Unit*) — la puce graphique : elle compose l'image, ligne par ligne.
- **L'APU** (*Audio Processing Unit*) — un mini-ordinateur sonore complet et autonome.
- **La mémoire** — 128 Ko de travail (WRAM), 64 Ko de vidéo (VRAM), et la cartouche (ROM + sauvegarde).
- **Le DMA** — un « déménageur » d'octets ultra-rapide entre ces mémoires.
- **Parfois, une puce dans la cartouche** — comme le SuperFX, un second processeur qui dessine de la 3D. Il y en a plusieurs familles, et elles changent la nature du problème (§10).

## 2. Petit rappel : bits, octets et hexadécimal

Pour comprendre la suite, trois notions suffisent. Tout, dans une console, se ramène à des **bits** — des interrupteurs qui valent 0 ou 1.

- Un **octet** (*byte*) = 8 bits. Il représente un nombre de 0 à 255.
- Un **mot** (*word*) 16 bits = 2 octets, de 0 à 65 535.
- On note souvent les nombres en **hexadécimal** (base 16), préfixés par **\$**. Chaque chiffre hexa vaut 4 bits. Ainsi **\$FF** = 255, **\$8000** = 32768.

Un octet valant **\$A9** (169), c'est ces huit interrupteurs :



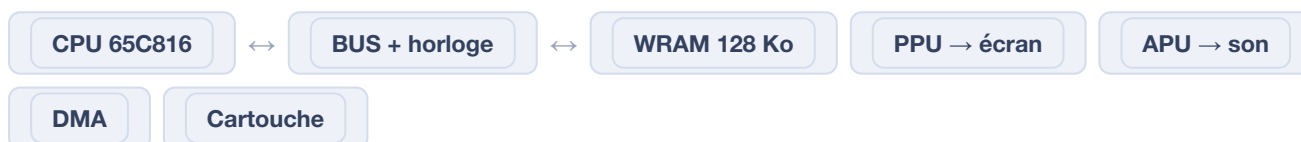
$128 + 32 + 8 + 1 = 169 = \$A9$ . (C'est aussi, par coïncidence, l'opcode de l'instruction « charge l'accumulateur » du 65C816 — on le reverra.)

#### 🔑 À retenir

Un **registre** est une petite case mémoire à l'intérieur d'une puce. Écrire une valeur dans un registre revient à **appuyer sur des boutons de commande** de la puce. La quasi-totalité du dialogue entre le CPU et les autres puces se fait ainsi : en écrivant des octets à des adresses spéciales appelées **registres MMIO**.

### 3. Vue d'ensemble : qui parle à qui

Tous les composants sont reliés par un **bus** : une autoroute d'adresses par laquelle le CPU lit et écrit. Quand le CPU accède à une adresse, ce n'est pas toujours de la mémoire — cela peut être un registre qui commande le PPU, l'audio, ou un transfert. Le rôle du bus est de **router** chaque accès vers le bon destinataire, et d'avancer l'horloge du bon nombre de cycles.



#### 🔑 À retenir

Dans le code de l'émulateur, cette structure se retrouve telle quelle : un module `cpu`, un module `ppu`, un module `apu`, un `bus` qui les relie, et un `scheduler` (l'ordonnanceur/horloge) qui garde tout le monde synchronisé. Le cœur d'émulation est une bibliothèque **sans aucune entrée/sortie** : elle se teste toute seule, sans écran ni son. Un programme séparé (le « frontend ») ouvre la fenêtre, joue le son et lit le clavier.

### La séparation cœur / frontend

Cette séparation est un principe d'ingénierie important. Le **cœur** ne connaît ni le clavier, ni la carte son, ni le système d'exploitation : il transforme simplement « appuis de manette + temps qui passe » en « image + échantillons audio ». Le **frontend** traduit cela vers le monde réel. Résultat : on peut faire tourner le cœur en mode « sans tête » (headless) pour le tester automatiquement — c'est exactement comme cela que cet émulateur a été vérifié (voir §16).

### 4. La cartouche et la carte mémoire

Le 65C816 adresse la mémoire sur **24 bits** : 16 Mo d'espace, découpé en 256 « banques » (*banks*) de 64 Ko. Une adresse s'écrit `banque:offset`, par exemple `$7E:0100`. Mais toutes les adresses ne mènent pas à de la RAM : selon la zone, un même accès touche la mémoire de travail, une puce, ou la cartouche.

|                    |                                             |
|--------------------|---------------------------------------------|
| \$00–\$3F : \$0000 | Miroir des 8 premiers Ko de la WRAM         |
| \$2100–\$213F      | Registres du PPU (vidéo)                    |
| \$2140–\$2143      | Ports de l'APU (son)                        |
| \$4200–\$421F      | Registres CPU internes (NMI, IRQ, manettes) |
| \$4300–\$437F      | Registres des 8 canaux DMA                  |
| \$8000–\$FFFF      | ROM de la cartouche                         |
| \$70–\$7D : \$0000 | Sauvegarde SRAM (pile de la cartouche)      |
| \$7E–\$7F : entier | 128 Ko de WRAM (mémoire de travail)         |

## LoROM et HiROM : deux câblages

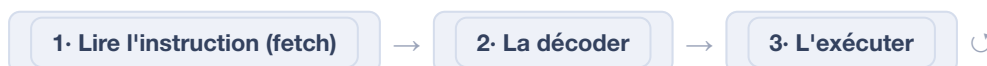
Les cartouches câblent leur ROM de deux façons principales. En **LoROM**, la ROM est exposée par blocs de 32 Ko dans la moitié haute ( \$8000–\$FFFF ) de chaque banque. En **HiROM**, elle occupe des banques pleines de 64 Ko. L'émulateur ne le sait pas d'avance : il lit l'**en-tête** interne de la ROM (un petit bloc de métadonnées à une position fixe) et attribue un **score** à chaque hypothèse — le couple somme de contrôle / complément doit valoir \$FFFF, le titre doit être lisible, l'octet de mapping doit être plausible — puis retient la meilleure. Il en déduit aussi la **région** (PAL 50 Hz ou NTSC 60 Hz) et la taille de la sauvegarde.

### Pour aller plus loin

**L'open bus, un détail subtil mais crucial.** Sur le vrai matériel, lire une adresse « qui ne mène nulle part » ne renvoie pas 0, mais **la dernière valeur qui a circulé sur le bus** — une sorte d'écho électrique du dernier octet transporté. Certains jeux en dépendent (volontairement ou par accident). L'émulateur mémorise donc cette dernière valeur dans un registre appelé **MDR** (*Memory Data Register*) et la renvoie pour toute adresse non mappée. Oublier ce détail provoque des bugs bizarres et lointains, très difficiles à diagnostiquer.

## 5. Le cœur : le processeur 65C816

Le CPU exécute la logique du jeu : « si le joueur appuie à droite, avancer Mario ». Il ne fait rien de magique — il répète sans cesse un cycle de trois temps, des millions de fois par seconde.



### Les registres du 65C816

Un registre est une case ultra-rapide interne au processeur. Le 65C816 en possède une poignée :

| Registre | Rôle                                                                                |
|----------|-------------------------------------------------------------------------------------|
| A        | Accumulateur — où se font les calculs (additions, comparaisons...).                 |
| X, Y     | Index — servent à parcourir des tableaux, compter des boucles.                      |
| S        | Pointeur de pile — mémorise où revenir après un sous-programme.                     |
| PC       | Compteur de programme — l'adresse de la prochaine instruction.                      |
| P        | Registre d'état — un octet de <i>drapeaux</i> (résultat nul ? négatif ? retenue ?). |
| DBR, PBR | Banques de données et de programme — les 8 bits hauts des adresses 24 bits.         |
| D        | Registre de page directe — une base pour un mode d'adressage rapide.                |

## La particularité 8/16 bits

Le 65C816 descend du 6502 (8 bits) qui équipait la NES. Il démarre en **mode émulation**, où il se fait passer pour un 6502, puis un jeu le bascule en **mode natif** 16 bits dès les premières instructions. Mieux : deux drapeaux du registre **P** (nommés **M** et **X**) permettent au jeu de choisir **à la volée** si l'accumulateur et les index travaillent sur 8 ou 16 bits. Une même instruction change donc de comportement selon le contexte — l'émulateur teste ces drapeaux à chaque opération concernée.

## Les modes d'adressage

Une instruction comme « charger l'accumulateur » a besoin de savoir *d'où* lire la valeur. Le 65C816 offre 24 **modes d'adressage** pour calculer cette adresse :

| Exemple      | Mode                 | Adresse calculée                                              |
|--------------|----------------------|---------------------------------------------------------------|
| LDA #\$1234  | Immédiat             | La valeur est dans l'instruction elle-même.                   |
| LDA \$2000   | Absolu               | Banque de données + <b>\$2000</b> .                           |
| LDA \$2000,X | Absolu indexé        | <b>\$2000</b> + contenu de X.                                 |
| LDA (\$10)   | Indirect             | Lit un pointeur en page directe, puis va à l'adresse pointée. |
| LDA [\$10],Y | Indirect long indexé | Pointeur 24 bits + Y — pour parcourir de gros tableaux.       |

## Le cycle d'exécution, en code

```
// Le squelette d'un pas de CPU (simplifié)
fn step(cpu, bus) {
    if une_interruption_est_en_attente() {
        servir_l_interruption(); // NMI (vsync) ou IRQ (timer)
        return;
    }
    let opcode = bus.read(cpu.pbr, cpu.pc); // 1. fetch
    cpu.pc += 1;
    match opcode { // 2. décode (256 cas)
        0xA9 => { let v = lire_immediat(cpu, bus); cpu.a = v; maj_drapeaux(cpu, v); } // LDA #
        0x8D => { let adr = lire_absolu(cpu, bus); bus.write(adr, cpu.a); } // STA abs
        0x4C => { cpu.pc = lire_mot(cpu, bus); } // JMP
        // ... les 253 autres opcodes ...
    }
}
```

Notre implémentation couvre les **256 instructions**, tous les modes d'adressage, l'arithmétique décimale (BCD, utilisée pour afficher scores et chronos), les instructions de pile et de saut, les déplacements de blocs mémoire, et les instructions spéciales comme **WAI** (met le CPU en veille jusqu'à la prochaine interruption) ou **STP** (arrêt total).

## Les interruptions

Une **interruption** est un signal qui force le CPU à suspendre ce qu'il fait pour exécuter un sous-programme spécial, puis reprendre. La SNES en a deux principales :

- **NMI** (*Non-Maskable Interrupt*) : déclenchée à chaque fin d'affichage d'image (le *V-blank*). C'est le **battement de cœur** du jeu — il y met à jour l'écran, lit les manettes, avance la musique. 50 ou 60 fois par seconde.
- **IRQ** : déclenchée à une ligne/position d'écran choisie par le jeu, pour des effets « à mi-écran » (une barre de statut fixe pendant que le décor défile, par exemple).

Servir une interruption suit une séquence précise : le CPU sauvegarde sur la pile la banque de programme, l'adresse courante et les drapeaux, met le drapeau d'interruption, puis saute à une adresse fixe (le **vecteur**) où le jeu a placé son gestionnaire. Reproduire cette séquence au cycle près est essentiel : beaucoup de jeux se calent précisément dessus.

### 🔑 À retenir

**Le temps se compte en cycles.** Chaque lecture ou écriture en mémoire fait avancer l'« horloge maître » d'un nombre précis de cycles, qui dépend de la zone visée : la ROM rapide, la RAM, les registres, la zone manettes... n'ont pas la même vitesse d'accès. C'est ainsi que l'émulateur sait *combien de temps* a passé, et donc quand déclencher les événements vidéo et audio.

## 6. Le chef d'orchestre : le timing et l'ordonnanceur

La SNES bat au rythme d'une horloge maître d'environ **21,5 MHz** (21,5 millions de cycles par seconde). L'image se construit ligne par ligne, comme un pinceau qui balaie l'écran de gauche à droite et de haut en bas.

|                    | NTSC (Amérique/Japon)             | PAL (Europe)  |
|--------------------|-----------------------------------|---------------|
| Horloge maître     | 21 477 272 Hz                     | 21 281 370 Hz |
| Lignes par image   | 262                               | 312           |
| Images par seconde | ≈ 60,1                            | ≈ 50,0        |
| Image visible      | 256 × 224 pixels (lignes 1 à 224) |               |

Chaque ligne dure 1364 cycles maîtres. Après la dernière ligne visible commence le **V-blank** : le « pinceau » remonte en haut de l'écran. C'est le seul moment sûr pour modifier la mémoire vidéo — d'où la NMI qui prévient le CPU à cet instant.

### L'ordonnanceur (scheduler)

Comment savoir, à tout moment, quel événement doit se produire (fin de ligne, NMI, IRQ, transfert automatique) sans tout ralentir ? L'émulateur utilise une astuce : il tient un seul compteur d'horloge (un entier 64 bits) et un unique « **prochain événement** ».

```
struct Scheduler {
    clock:      u64,    // cycles écoulés depuis l'allumage
    next_event: u64,    // horodatage du prochain événement
    v:          u16,    // ligne courante (0..261 ou 0..311)
}
```

À chaque accès mémoire du CPU, on ajoute le coût en cycles, puis on compare l'horloge à `next_event`. Tant que l'horizon n'est pas franchi, on ne fait rien — une seule comparaison. Quand il l'est, on traite l'événement (dessiner la ligne, déclencher NMI/IRQ, faire avancer l'audio) et on calcule le prochain horizon. Ce mécanisme est ce qui rend l'émulation à la fois **exacte** et **rapide**.

#### Pour aller plus loin

La SNES supporte deux régions : PAL (Europe, 50 Hz) et NTSC (60 Hz). Un jeu européen tourne plus lentement mais affiche plus de lignes. L'émulateur détecte la région dans l'en-tête de la cartouche et adapte l'ordonnanceur et la cadence d'affichage : une cartouche européenne est rendue à 50 images par seconde, exactement comme sur une SNES européenne branchée à une télé de l'époque, et une cartouche japonaise ou américaine à 60.



## 7. La puce graphique : le PPU

Le PPU est le composant le plus riche — et le plus délicat à émuler fidèlement. Son rôle : transformer des **tuiles** et des **sprites** rangés en mémoire vidéo en une image de 256 × 224 pixels, soixante fois par seconde.

### Trois mémoires vidéo

- **VRAM** (64 Ko) : les *tuiles* (petits carrés de 8×8 pixels) et les *plans de tuiles* (quelle tuile à quel endroit).
- **CGRAM** (256 couleurs) : la palette. Chaque pixel ne stocke pas une couleur, mais un *numéro* de couleur dans cette palette. Chaque couleur est codée sur 15 bits (5 bits de rouge, 5 de vert, 5 de bleu).
- **OAM** (544 octets) : la table des sprites — position, tuile, priorité de chacun des 128 objets mobiles.

#### 💡 Analogie

Imaginez un tableau de feutrine avec des gommettes. Les **tuiles** sont un stock de petits motifs réutilisables ; le **plan de tuiles** dit où coller chaque motif ; la **palette** dit quelles couleurs donner aux motifs. Redessiner l'écran, c'est relire cette recette 50 à 60 fois par seconde. Réutiliser les tuiles économise énormément de mémoire — indispensable en 1990.

### Comment une tuile est stockée : les plans de bits

Une tuile 4 bits (16 couleurs) fait 8×8 pixels, chacun un numéro de 0 à 15. Ce numéro est découpé en 4 **plans de bits** : le plan 0 contient le bit de poids faible de chaque pixel, le plan 1 le bit suivant, etc. Pour reconstruire un pixel, on rassemble un bit de chaque plan :

```
// pixel (x=0) d'une ligne, à partir de 4 octets de plans
val = (plan0 >> 7 & 1)
    | (plan1 >> 7 & 1) << 1
    | (plan2 >> 7 & 1) << 2
    | (plan3 >> 7 & 1) << 3; // -> numéro de couleur 0..15
```

Se tromper dans l'ordre des plans ou des bits produit une image « en rayures » caractéristique — un piège classique. Cet émulateur a justement rencontré ce symptôme en développant le SuperFX (voir §10).

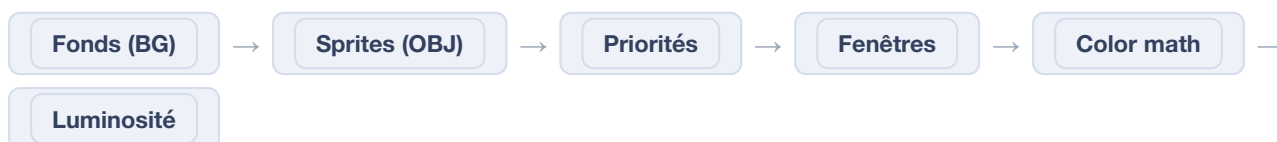
### Les couches de fond et les huit modes

Le PPU superpose jusqu'à 4 **couches de fond** (BG1–BG4). Un registre ( \$2105 ) choisit le **mode** (0 à 7), qui décide combien de couches existent et avec combien de couleurs :

| Mode | Couches et profondeur                        | Usage typique                      |
|------|----------------------------------------------|------------------------------------|
| 0    | 4 couches, 4 couleurs chacune                | Menus simples                      |
| 1    | BG1/BG2 16 couleurs, BG3 4 couleurs          | Le plus courant (plateformes, RPG) |
| 2    | 2 couches 16 couleurs + décalage par colonne | Effets de profondeur               |
| 3    | BG1 256 couleurs, BG2 16 couleurs            | Écrans riches (menus, titres)      |
| 5/6  | Haute résolution (512 points de large)       | Textes fins, écrans de saisie      |
| 7    | 1 couche, rotation + zoom                    | Mode 7 : circuits, cartes du monde |

## Le rendu ligne par ligne

Notre PPU dessine **une ligne à la fois**, à la fin de chaque ligne visible. C'est essentiel : comme le jeu peut changer les registres vidéo *entre* deux lignes, dessiner ligne par ligne reproduit naturellement presque tous les effets d'écran. Pour chaque ligne, le pipeline est :



### Sprites (OBJ)

Jusqu'à 128 objets mobiles, de tailles variées. Le PPU a des limites matérielles strictes : au plus **32 sprites** et **34 tuiles par ligne**. Au-delà, les sprites en trop disparaissent — un phénomène de clignotement que les joueurs de l'époque connaissaient bien, et que l'émulateur reproduit fidèlement (drapeaux de dépassement inclus).

### Priorités

Les **priorités** décident qui passe devant qui — Mario devant le décor mais derrière un nuage. Chaque mode a sa propre table d'ordre de superposition entre les couches et les 4 niveaux de priorité des sprites. Une erreur ici et les personnages passent derrière le décor.

### Fenêtres et color math

Les **fenêtres** masquent des zones rectangulaires — le rond de lumière d'une torche dans une pièce sombre. La **color math** additionne ou soustrait deux « écrans » (principal et secondaire) pour créer transparences, ombres et fondus. Par exemple, une eau semi-transparente : chaque canal de couleur est moyenné entre le décor et l'eau. C'est cette color math qui allume le menu de sélection de *Super Mario All-Stars* (superposition de deux couches) — sans elle, l'écran resterait noir.

### Le Mode 7 : rotation et zoom

Le Mode 7 est à part : une seule couche, mais qu'il transforme par une matrice mathématique appliquée à **chaque pixel**. Pour le pixel écran (x, y), la position dans la texture est :

```

tex_x = A · (x + H) + B · (y + V) + X0
tex_y = C · (x + H) + D · (y + V) + Y0
  
```

où A, B, C, D forment la matrice (en virgule fixe 8.8), et H/V/X0/Y0 les décalages et le centre. En faisant varier la matrice à **chaque ligne** (via le HDMA, §8), on obtient la perspective des circuits de F-Zero : le sol semble filer vers l'horizon. Dans *Super Mario Kart*, cette matrice n'est même pas calculée par la console : une puce de la cartouche s'en charge (§10).

#### Pour aller plus loin

**Fonctionnalités fines.** Le PPU gère aussi : la *mosaïque* (pixellisation pour les transitions), le *décalage par tuile* (offset-per-tile, où une couche fournit un décalage de défilement différent pour chaque colonne de 8 pixels), la *haute résolution* (512 points, pour les textes fins des écrans de saisie de nom), et l'*entrelacement* (448 lignes). Chacune est un petit monde de règles que l'émulateur reproduit d'après la documentation matérielle.

## 8. Le déménageur : DMA et HDMA

Copier des milliers d'octets vers la VRAM un par un avec le CPU serait trop lent. Le **DMA** (*Direct Memory Access*) le fait à la place, en bloc, à environ un octet tous les 8 cycles. Le jeu remplit quelques registres (adresse source, registre de destination, taille) puis lance le transfert ; le CPU est mis en pause le temps de la copie. La SNES a **8 canaux** DMA indépendants.

Le **HDMA** (*H-blank DMA*) est plus malin : il transfère **un petit paquet à chaque ligne d'écran**, automatiquement, juste avant que la ligne soit dessinée. C'est ce qui permet :

- les **dégradés de ciel** — une couleur de fond différente par ligne ;
- la **perspective du Mode 7** — une matrice différente par ligne ;
- les **écrans ondulants** — un décalage horizontal par ligne (effet de chaleur, de vague).

#### À retenir

Détail qui fait tout : le HDMA d'une ligne doit s'exécuter **avant** le rendu de cette ligne. Se tromper d'une ligne, et le dégradé est décalé d'un cran. C'est l'une des sources de bugs les plus classiques d'un émulateur — d'où l'attention portée à l'ordre exact des événements dans l'ordonnanceur.

## 9. Le son : l'APU (SPC700 + S-DSP)

Surprise : la partie audio de la SNES est **un ordinateur à part entière**, avec son propre processeur (le **SPC700**), sa propre mémoire (64 Ko) et sa puce de synthèse (le **S-DSP**). Il ne partage rien avec le CPU principal, sauf **quatre boîtes aux lettres** (les ports \$2140–\$2143).

#### Analogie

Le CPU principal et l'APU sont comme deux bureaux qui ne communiquent que par quatre cases postales. Au démarrage, un minuscule programme gravé dans l'APU (l'*IPL ROM*, 64 octets) répond « je suis prêt » (les octets \$AA, \$BB) ; le jeu lui envoie alors, case par case, tout son **moteur sonore** et ses instruments. La quasi-totalité des jeux *attendent* cette poignée de main avant de continuer — l'émuler est donc indispensable, même pour afficher la moindre image.

## Comment naît un son

Le S-DSP mélange **8 voix** en un flux stéréo à 32 kHz. Pour chaque voix :

- **BRR** : les instruments sont des échantillons sonores compressés en blocs de 9 octets (le format *Bit Rate Reduction*). On les décompresse à la volée, en appliquant un petit filtre qui reconstruit la forme d'onde à partir des deux échantillons précédents.
- **Hauteur (pitch)** : on lit l'échantillon plus ou moins vite pour jouer une note grave ou aiguë, avec une interpolation gaussienne pour adoucir le résultat (une table de 512 coefficients).
- **Enveloppe ADSR** : le volume évolue dans le temps selon quatre phases — *Attack* (montée), *Decay* (chute), *Sustain* (maintien), *Release* (extinction). C'est ce qui distingue un piano (attaque vive, extinction lente) d'un orgue (volume constant).
- **Bruit** : un générateur pseudo-aléatoire, pour les explosions et vagues.
- **Écho** : un tampon circulaire en mémoire, relu avec un délai et un filtre à 8 coefficients, recrée la réverbération si caractéristique des musiques de la SNES — pensez aux grottes de *Secret of Mana*.

### Pour aller plus loin

**Synchronisation et son de l'ordinateur.** L'émulateur fait tourner le SPC700 « en rattrapage » : le bus le fait avancer juste avant chaque accès aux boîtes aux lettres et une fois par ligne d'écran, avec une conversion d'horloge en virgule fixe (32.32 bits) pour ne jamais dériver. Les échantillons à 32 kHz produits sont ensuite **ré-échantillonnés** vers la fréquence de la carte son de l'ordinateur (souvent 44,1 ou 48 kHz), avec un léger ajustement dynamique du débit pour absorber la différence entre l'horloge émulée et celle du PC. Résultat : une éventuelle dérive devient un imperceptible vibrato plutôt qu'un craquement. La fidélité audio a d'ailleurs été vérifiée en analysant le signal produit (énergie, présence de l'écho) — pas seulement à l'oreille.

## 10. Les puces de cartouche : quatre processeurs de plus

Voici le point où « émuler la Super Nintendo » cesse d'être une phrase suffisante. La console expose sur son connecteur de cartouche assez de signaux pour qu'un éditeur y branche **sa propre puce** — et plusieurs l'ont fait, chacun la sienne. Une cartouche n'est donc pas toujours une simple mémoire morte : c'est parfois un ordinateur de plus, avec son processeur, sa mémoire et ses règles. Émuler *la console* ne suffit pas ; il faut émuler *cette cartouche-là*.

### À retenir

Conséquence directe pour qui écrit un émulateur : la compatibilité n'est pas une courbe continue qu'on ferait monter en corrigeant des détails. Chaque puce de cartouche est une **marche**. Tant qu'elle n'est pas écrite, les jeux qui l'utilisent ne démarrent pas du tout ; le jour où elle l'est, ils démarrent tous ensemble. Cet émulateur en implémente quatre.

| Puce          | Ce qu'elle est                                                      | Approche                          | Jeu de validation       |
|---------------|---------------------------------------------------------------------|-----------------------------------|-------------------------|
| SuperFX (GSU) | Processeur RISC 16 bits ~10 MHz qui dessine des pixels              | CPU complet, écrit de zéro        | <i>Yoshi's Island</i>   |
| SA-1          | Un second 65C816, cadencé à la moitié de l'horloge maître           | CPU complet, réutilisant le nôtre | <i>Super Mario RPG</i>  |
| DSP-1         | Calculateur NEC $\mu$ PD77C25 : matrices, projection, trigonométrie | Haut niveau (HLE)                 | <i>Super Mario Kart</i> |
| CX4           | Hitachi HG51B169 (Capcom) : vecteurs, rotation/zoom de sprites      | Haut niveau (HLE)                 | <i>Mega Man X3</i>      |

## Deux façons d'émuler une puce, et pourquoi le choix est imposé

Le SuperFX et le SA-1 sont des **processeurs programmables** : le programme qu'ils exécutent est dans la cartouche, donc dans le fichier que possède le joueur. On peut — et on doit — écrire le processeur lui-même, instruction par instruction, et le laisser exécuter ce programme. C'est ce qu'on appelle l'émulation **bas niveau**.

Le DSP-1 et le CX4 n'offrent pas ce luxe. Leur programme interne est gravé dans un *micrologiciel* qui n'est pas dans la cartouche et qui reste la propriété de son éditeur. Impossible de l'exécuter ; il faut se contenter d'observer le **dialogue** — le jeu écrit un numéro de commande et des paramètres, la puce rend un résultat — et de réimplémenter chaque commande par ses mathématiques. C'est l'émulation **haut niveau** (HLE) : on ne rejoue pas la puce, on rejoue ce qu'elle répond.

### 🔑 À retenir

Le HLE a un défaut assumé qu'il faut connaître : il est exact **pour les commandes que les jeux utilisent**, telles qu'on les a comprises. Une commande rare, ou un jeu qui s'appuierait sur un effet de bord non documenté, sortirait du modèle. En contrepartie, il est instantané : notre DSP-1 déclare toujours « résultat prêt » (bit 7 de son registre d'état), là où la vraie puce aurait fait patienter le jeu le temps de son calcul. Aucun jeu testé ne s'en plaint — mais c'est une hypothèse, pas un théorème.

## Le SuperFX (GSU) : écrire un processeur complet de plus

La plus célèbre de ces puces est le **SuperFX** (aussi appelé GSU) : c'est elle qui faisait tourner la 3D de *Star Fox* et les décors de *Yoshi's Island*.

Le GSU est un processeur **RISC 16 bits** à ~10 MHz, avec 16 registres et un jeu d'instructions entièrement différent du 65C816. Il vit dans la cartouche, accède à la ROM et à une RAM dédiée (*Game Pak RAM*), et sait **dessiner des pixels** (instruction **PL0T**) dans un tampon que le jeu récupère ensuite pour l'afficher. Le CPU principal et le GSU se partagent la ROM/RAM à tour de rôle : pendant que le GSU calcule, le CPU attend.

L'émuler, c'est écrire **un CPU complet de plus** — le premier grand chantier de ce genre dans ce projet. Le GSU de cet émulateur exécute réellement les programmes de *Yoshi's Island*, décompresse ses graphismes et affiche l'écran de sélection de la langue.

### Histoire d'un bug

**Un octet qui changeait tout.** Pendant des jours, *Yoshi's Island* restait obstinément noir : le GSU tournait, mais son décompresseur bouclait sans fin. Après avoir construit un outil de trace dédié et éliminé une à une plusieurs pistes (format des pixels, décodage des banques mémoire, tampon de lecture), la cause s'est révélée être un **décalage d'un seul octet sur le compteur de programme R15** : notre code avançait R15 dès la lecture de l'instruction, alors que le vrai GSU ne l'avance qu'à la lecture des opérandes. Toutes les valeurs calculées à partir de R15 (cibles de saut, adresses) étaient donc fausses d'un octet, ce qui donnait de mauvais compteurs de boucle et figeait la décompression. Une ligne de correction, et le jeu s'est mis à booter. C'est l'essence du travail d'émulation : des bugs minuscules aux conséquences spectaculaires, que seule une comparaison méticuleuse avec le matériel de référence permet de trouver.

## Le SA-1 : une seconde console dans la cartouche

Le **SA-1** (*Super Accelerator 1*) prend le problème par l'autre bout. Plutôt qu'un processeur exotique, Nintendo a remis dans la cartouche **le même processeur que dans la console** — un 65C816 — mais cadencé à la moitié de l'horloge maître, soit **10,74 MHz**. Le 65C816 de la console, lui, ne dépasse pas 3,58 MHz, même sur ses accès les plus rapides : la cartouche est donc environ **trois fois plus rapide que la machine dans laquelle on l'enfonce**. C'est la puce de *Super Mario RPG*.

Pour l'émulateur, c'est une bonne nouvelle inhabituelle : le processeur est déjà écrit. Le module `cpu` du §5 est simplement **instancié une seconde fois**. Tout le travail restant est autour : le *Super MMC* (un contrôleur qui redécoupe la ROM en fenêtres réattribuables), une unité arithmétique matérielle (multiplication, division, accumulation), un lecteur de flux de bits à longueur variable pour les décompressions, une minuterie H/V, un contrôleur DMA, 2 Ko de mémoire interne partagée avec la console (I-RAM) et jusqu'à 256 Ko de mémoire sauvegardée par pile (BW-RAM).

### Histoire d'un bug

**Deux processeurs qui s'attendent l'un l'autre.** *Super Mario RPG* ne démarrait pas : il figeait à l'écran noir. La trace, ajoutée pour l'occasion ( `--trace-sa1` , jumelle de celles du 65C816 et du SPC700), montrait deux boucles parfaitement immobiles — le SA-1 relisait sans fin l'adresse `$40:3D00` , et le processeur de la console relisait sans fin l'octet `$3002` de l'I-RAM. Chacun attendait un signal que l'autre devait poser.

La cause tenait dans une condition trop stricte. La BW-RAM possède un verrou d'écriture ( `SBWE` , registre `$2226` ) ; notre code refusait toute écriture venue de la console quand ce verrou était fermé. Le vrai matériel est plus subtil : il ne refuse l'écriture que si le verrou est fermé **et** que l'adresse tombe dans la zone explicitement protégée. Or le jeu écrivait `$18` en `$40:3D00` , verrou fermé mais *hors* de la zone protégée — une écriture parfaitement légale, que nous jetions. Le signal attendu par le SA-1 n'arrivait donc jamais, et l'attente devenait mutuelle.

Un seul `et` manquant dans une condition ; un jeu entier qui ne démarre pas. Une fois corrigé, *Super Mario RPG* enchaîne l'écran « HELP! », la plaine forestière jouable, puis la cinématique du haricot.

## Le DSP-1 : la perspective de *Super Mario Kart*

Le **DSP-1** est un calculateur, pas un processeur généraliste : un NEC  $\mu$ PD77C25 auquel le jeu pose des questions de mathématiques. La console lui parle par deux guichets d'un octet — **DR** pour les données, **SR** pour l'état — et le protocole est d'une simplicité désarmante : écrire un numéro de commande, écrire les paramètres, relire le résultat.

Ce que la puce calcule, en revanche, est exactement ce dont on parlait au §7. Les commandes utilisées par *Super Mario Kart* forment une petite chaîne de rendu 3D :

| Commande | Nom       | Ce qu'elle fait                                                                         |
|----------|-----------|-----------------------------------------------------------------------------------------|
| 0p02     | Parameter | Installe la caméra : hauteur, angle de vue, centre de l'écran.                          |
| 0p0A     | Raster    | <b>Rend les quatre coefficients A, B, C, D du Mode 7 pour une ligne d'écran donnée.</b> |
| 0p06     | Project   | Projette un point du monde (x, y, z) en coordonnées d'écran (H, V) + une échelle.       |
| 0p0E     | Target    | L'inverse : d'un point de l'écran vers le point du sol qu'il désigne.                   |

Relisez la ligne **0p0A** en gardant en tête le §7 et le §8, et tout le circuit apparaît : la matrice *par ligne* qui donne au Mode 7 sa perspective, c'est le DSP-1 qui la calcule, le HDMA qui la dépose dans les registres du PPU juste avant que la ligne soit dessinée, et le PPU qui l'applique pixel par pixel. Trois chapitres de ce document se rejoignent dans un seul virage de *Super Mario Kart*.

### Pour aller plus loin

**Pourquoi c'était la partie risquée.** Tout ce calcul est en virgule fixe Q15 — des entiers 16 bits interprétés comme des fractions — avec des normalisations, des exposants et des saturations à chaque étape. Un test unitaire vérifie qu'une fonction rend le nombre attendu, mais il ne sait pas dire si la *route* a l'air d'une route. Une erreur d'un bit sur un arrondi de division ne casse rien : elle courbe l'horizon. La seule preuve acceptable était donc de conduire réellement — et le circuit s'affiche correctement, chronomètre qui avance, sur une vraie course.

## Le CX4 : la puce dessinée pour un seul studio

Le **CX4** est un Hitachi HG51B169 que Capcom a mis dans *Mega Man X2* et *X3*. Il fait du fil de fer vectoriel, de la rotation et du zoom de sprites, de la trigonométrie, et sait même construire directement la table des sprites (l'OAM du §7) à la place du jeu. Il s'expose de façon minimaliste : une plage plate de 8 Ko en **\$6000–\$7FFF**, où le jeu dépose ses paramètres, déclenche une commande, et relit le résultat au même endroit. Notre implémentation est HLE, pour la même raison que le DSP-1 : le microprogramme 24 bits de la puce appartient à Capcom.



### Histoire d'un bug

**Un compteur qui passe sous zéro.** *Mega Man X3* démarrait, affichait son écran-titre — dont l'animation du logo est justement calculée par le CX4 — puis **plantait** juste après l'introduction : pas un écran figé, un véritable arrêt de l'émulateur. Le coupable était une boucle de nettoyage de la table des sprites, écrite ainsi : *tant que l'index est supérieur au début, effacer et reculer de quatre*. L'index descend par pas de 4 sans être aligné sur la borne : il la *franchit* au lieu de s'y arrêter. Et quand la borne valait moins de 4, la soustraction s'appliquait à un entier non signé, qui n'a pas de « moins un » : la valeur remontait d'un coup à environ 18 milliards de milliards, et l'accès suivant sortait de la mémoire.

C'est un bug instructif à double titre. D'abord parce qu'il est **importé** : la logique vient d'une réimplémentation de référence écrite en C, où le même dépassement se serait produit en silence, en corrompant des octets voisins — un jeu qui devient bizarre plus tard, sans rien pour désigner la cause. Rust refuse de continuer, et transforme une corruption discrète en arrêt net à l'instruction fautive. Ensuite parce que le correctif est d'une banalité totale (soustraire prudemment, sortir de la boucle si l'on passerait sous zéro) alors que le symptôme était spectaculaire. Une fois corrigé, le jeu franchit l'intro et tourne 12 000 images scriptées sans incident.

### À retenir

**Ce que les quatre ont en commun.** Aucune de ces puces ne *possède* la ROM de la cartouche : c'est la cartouche qui la leur prête à chaque appel. C'est un choix d'architecture, pas un détail — une seule copie de la ROM en mémoire, aucun risque que deux composants aient deux versions différentes de la même cartouche, et un état sauvegardable qui ne contient jamais l'image du jeu. Et toutes suivent la même règle de synchronisation que le processeur audio (§9) : on ne les fait pas avancer en parallèle, on les fait **rattraper** le temps écoulé quand quelqu'un vient leur parler. Le résultat reste déterministe, ce qui est la condition de tout le reste (§13).

## 11. Pourquoi « au cycle près » : l'affaire Terranigma

Le §1 affirme qu'un émulateur doit reproduire non seulement *ce que* font les puces mais *quand* elles le font. C'est une phrase facile à écrire et difficile à faire sentir. Voici donc, en détail, le cas où elle s'est vérifiée dans ce projet — parce qu'il montre exactement à quoi ressemble « un jeu qui dépend d'une marge non documentée ».

### Le symptôme

*Terranigma*, un action-RPG de 1995, était **muet**, et se **figeait** après la carte d'annonce « Chapitre 1 ». Deux défauts apparents ; un seul bug. La trace du processeur principal était sans appel : sur 607 954 instructions enregistrées, **100 %** étaient les deux mêmes, en boucle — `LDA $2140 / BNE`. Le jeu attendait une réponse de son moteur sonore. Le moteur sonore, lui, avait déraillé jusqu'à l'adresse `$0000` et exécutait la page zéro.



## La règle qui était fausse

Une instruction du SPC700 dure de 2 à 12 cycles, et fait son **accès mémoire sur son dernier cycle**. Notre code faisait l'inverse : dès qu'un cycle de l'instruction était dû, il l'exécutait entièrement puis facturait le coût. Autrement dit, tous les effets d'une instruction — y compris l'écriture dans les boîtes aux lettres `$F4-$F7` par lesquelles l'APU répond à la console — devenaient visibles **jusqu'à 62 cycles maîtres trop tôt** pour une instruction de 4 cycles.

## Pourquoi six autres jeux ne voyaient rien

C'est la partie qui vaut le détour. Presque tous les jeux envoient leur moteur sonore en passant par la **routine d'amorçage gravée dans la console** (l'IPL du §9). Or cette routine lit l'octet de données *avant* d'accuser réception :

```
// IPL, à $FFDD – lit, PUIS accuse
CMP  Y,$F4      // attendre le numéro de séquence
MOV  A,$F5      // LIRE la donnée
MOV  $F4,Y      // puis seulement : accuser réception
```

Dans cet ordre, aucune course n'est possible : quand la console voit l'accusé, la donnée est déjà lue. Un accusé publié 62 cycles trop tôt ne change strictement rien. Six jeux passaient donc à travers le bug sans le moindre symptôme.

*Terranigma* n'utilise pas l'IPL. Il installe **son propre récepteur**, à l'adresse `$041D`, et celui-ci accuse **d'abord** :

```
// récepteur de Terranigma, à $041D – accuse, PUIS lit
MOV  $F4,Y      // accuser réception
MOV  A,$F5      // puis lire la donnée
```

Cet ordre n'est pas une erreur du programmeur : il est plus rapide, et il est **sûr sur le vrai matériel**, parce que le SPC700 dispose là d'environ **14 cycles maîtres** de marge réelle entre l'accusé et la lecture — assez pour que la console n'ait pas le temps de réécrire le port. Notre erreur de 62 cycles a **inversé** cette marge. La console écrasait `$2141` avant que l'APU n'ait lu la valeur précédente.

### À retenir

Le jeu ne fait rien d'illégal. Il s'appuie sur une propriété du silicium que personne n'a jamais écrite dans une documentation, parce qu'elle allait de soi pour qui avait la puce sous les yeux. **C'est cela, « un jeu qui dépend du timing »** : pas un caprice, une marge de quatorze cycles.

## Les dégâts, et le correctif

Le résultat de la course perdue était mécanique : **2 743 octets faux sur les 7 316 transférés**. Le moteur sonore arrivait corrompu, son pointeur de séquence finissait par valoir `$0000`, et il partait exécuter la page zéro. D'où le silence — et d'où le gel, puisque la console attendait indéfiniment une réponse d'un programme qui n'existait plus.

Le correctif ne comporte ni exception ni cas particulier : une table des durées de chaque instruction SPC700, et un ordonnanceur qui **refuse de commencer une instruction tant que le budget de temps**

**ne couvre pas sa fenêtre entière.** Le rythme moyen du processeur audio est inchangé ; seul bouge le moment, à *l'intérieur* de sa fenêtre, où une instruction publie ses effets — de son premier cycle vers son dernier, là où le matériel fait réellement son accès au bus.

| Mesure sur <i>Terranigma</i>            | Avant                    | Après                                                 |
|-----------------------------------------|--------------------------|-------------------------------------------------------|
| Amplitude audio maximale (900 images)   | 0                        | 5 038                                                 |
| Volume général de l'APU ( MVOL )        | \$00                     | \$5F                                                  |
| Mémoire APU réellement remplie          | 5 334 o                  | 60 611 o                                              |
| Octets faux au transfert                | 2 743 / 7 316            | 0 / 2 387                                             |
| Progression depuis un démarrage à froid | gel après « Chapitre 1 » | « Chapitre 1 » à l'image 2 000, chambre d'Ark à 4 800 |

### À retenir

**La leçon, et c'est elle qui compte.** Une règle de timing fausse ne casse pas « un jeu » : elle casse **une classe de jeux** — ici, tous ceux qui installent leur propre récepteur au lieu d'utiliser celui de la console. Et sa correction ne répare pas qu'un jeu : elle répare toute la classe, y compris les jeux qu'on n'a pas encore essayés. C'est pourquoi on ne corrige jamais ce genre de défaut par un ajustement empirique jusqu'à ce qu'un titre passe. Les sept autres jeux de test ont vu leur amplitude audio bouger de  $\pm 1\%$  — chaque instruction SPC atterrit désormais une fenêtre plus loin — et aucun n'a perdu son son.

### Pour aller plus loin

**Une honnêteté à conserver.** Une chose n'a jamais été établie : lors d'une vérification intermédiaire, une capture semblait montrer le bug encore présent. Deux lectures restent compatibles avec les faits — soit cette capture venait de l'ancien binaire, soit la même course se déclenchait par intermittence. Quatre tentatives de reproduction ont échoué, et le doute n'a pas été levé ; il est consigné plutôt que tu.

Il en reste aussi une leçon de méthode, coûteuse à réapprendre : **installer un binaire ne change rien à un processus déjà lancé.** Un fichier écrit après une installation peut parfaitement provenir de l'ancien code. On compare les horodatages au *processus*, pas au fichier sur le disque.

## 12. La boucle d'émulation : tout synchroniser

Assembler tout cela tient, au fond, en une boucle. Pour produire une image :

```
fn run_frame(inputs) -> Image {
    appliquer_les_manettes(inputs);
    while !fin_de_trame {
        cpu.step(bus); // le CPU exécute une instruction...
        // ...chaque accès mémoire a fait avancer l'horloge, ce qui
        // déclenche AU BON MOMENT, via l'ordonnanceur :
        // • le rendu d'une ligne d'écran,
        // • les interruptions NMI / IRQ,
        // • les transferts HDMA,
        // • le rattrapage du processeur audio.
    }
    return framebuffer; // l'image de 256x224 est prête
}
```

Le frontend appelle `run_frame` à la cadence de la région (50 ou 60 fois par seconde), convertit l'image en pixels à l'écran (via une fenêtre `winit` et la bibliothèque `pixels`), envoie le son à la carte audio (`cpal`), et transmet l'état du clavier comme s'il s'agissait d'une manette. Le rythme est tenu par une **échéance absolue** : on calcule l'instant idéal de la prochaine image et on attend jusque-là, pour un défilement parfaitement régulier.

## 13. Au-delà de la console : sauvegardes et save states

Parce que l'émulateur **possède la totalité de l'état** de la machine, il peut faire des choses impossibles sur le vrai matériel :

- **Sauvegardes de cartouche (SRAM)** : la pile de sauvegarde de la vraie cartouche est simplement un fichier `.srm` à côté de la ROM, chargé au lancement et réécrit en quittant.
- **Save states** : on *séréalise* l'intégralité de l'état — CPU, PPU, APU, toutes les mémoires — dans un fichier. Le recharger fait reprendre le jeu **exactement** où il en était, au pixel et à l'échantillon près. Cette exactitude est vérifiée automatiquement : après sauvegarde puis rechargement, l'émulation reste identique bit pour bit.
- **Outils de débogage** : traces d'instructions, captures d'écran, inspection de la mémoire — les instruments qui ont permis de construire et de valider l'émulateur.

### Pour aller plus loin

**Le déterminisme, garant de justesse.** Un émulateur bien écrit est *déterministe* : à partir du même état et des mêmes entrées, il produit toujours exactement la même suite d'images et de sons. C'est cette propriété qui rend les save states fiables, et qui permet de tester l'émulateur en comparant ses sorties à une référence connue.

## 14. L'application autour de l'émulateur

Tout ce qui précède décrit une **bibliothèque** : un composant sans écran, sans son et sans clavier, qui transforme « appuis de manette + temps qui passe » en « image + échantillons ». Ce n'est pas encore un logiciel qu'on ouvre. Autour d'elle a poussé une application, et ses parties

les plus intéressantes ne sont justement pas de l'émulation : elles exploitent des propriétés que *seul* un émulateur possède.

## La bibliothèque, et l'identification par empreinte

L'écran d'accueil est une grille de jeux. Chaque carte a une image — et cette image, personne ne l'a téléchargée : l'application **émule le jeu quelques centaines d'images sur un fil d'arrière-plan** et garde le résultat. Un émulateur est le seul logiciel de bibliothèque qui puisse fabriquer lui-même la vignette de ce qu'il liste.

Reste tout ce qu'une vignette ne dit pas : le genre, l'éditeur, l'année, le nombre de joueurs, la jaquette d'origine. La tentation évidente est de chercher par nom de fichier — et c'est précisément l'erreur, parce que chercher par nom, c'est **deviner** : `Super Mario Kart (E) [!].zip` ressemble à beaucoup de choses. La devinette est supprimée en passant par une **empreinte** :



Le CRC32 est une signature de 4 octets calculée sur les octets du jeu : deux fichiers de noms différents mais de contenu identique la partagent, deux versions différentes ne la partagent jamais. Une base de correspondances publique la traduit en **nom canonique** — *Super Mario Kart (Europe)* — de façon certaine et non par ressemblance. Les catalogues de faits sont ensuite indexés sur ce même CRC : genre (3 851 entrées pour la SNES), développeur (3 850), éditeur (3 845), nombre de joueurs (3 759), année (3 305), classification d'âge (1 156). La jaquette, elle, est nommée d'après le nom canonique — fiable puisque ce nom vient du CRC.

### Pour aller plus loin

**Là où la chaîne redevient faillible, on le dit.** Aucune de ces bases ne contient la moindre *description* : vérifié en listant l'arborescence complète. Le seul texte disponible sans compte vient de Wikipédia, dont l'interrogation se fait **par titre** — et donc par ressemblance, avec le risque de tomber sur l'homonyme. Trois conséquences assumées plutôt que découvertes : la description est **visiblement attribuée** à Wikipédia, pour qu'une erreur se voie et s'explique au lieu de passer pour une affirmation de l'application ; sa licence impose de toute façon cette attribution ; et elle est signalée comme **anglophone** dans une interface qui, elle, est bilingue.

Le réseau, enfin, n'est jamais sollicité tout seul : rien au démarrage, rien au balayage du dossier. Deux boutons explicites, et tout ce qui a été récupéré est gardé sur le disque. Les fichiers de catalogue se téléchargent *une fois* et servent ensuite hors ligne à toute la collection — un aller-retour pour des milliers de jeux, ce qui est aussi la façon la plus sobre de traiter des serveurs qu'on ne paie pas.

## Deux langues, vérifiées par le compilateur

L'interface existe en français et en anglais. La méthode est plus intéressante que le résultat. Plutôt qu'un fichier de traductions — où un oubli ne se voit qu'à l'écran, trop tard — les deux langues sont déclarées **côte à côte**, et le code engendré couvre les deux :

```
messages! {
  SettingsDisplay => "Affichage" / "Display",
  LibraryEmpty   => "Aucun jeu" / "No games",
}
```

Un message ajouté sans son anglais **ne compile pas**. C'est tout l'intérêt : transformer un oubli d'humain en erreur de compilation. Deux règles de bon sens complètent cela — les noms de touches restent en anglais dans les deux langues (une touche se nomme par ce qui est gravé dessus), et l'anglais étant 15 à 20 % plus court que le français, **aucune largeur de colonne n'est calculée sur une chaîne codée en dur** : on mesure le texte réellement rendu, dans la langue active.

## Une manette dessinée qui est aussi un testeur

L'écran des commandes montre, à côté du tableau des affectations, une **manette SNES dessinée au trait** (au vecteur, donc sans aucune image à charger). Elle n'est pas décorative : survoler une ligne du tableau met en évidence le bouton correspondant sur le dessin, cliquer un bouton du dessin lance sa reconfiguration, et surtout **les boutons réellement pressés s'allument**. Cela fait de cet écran, gratuitement, le moyen le plus rapide de découvrir qu'une manette est à moitié morte — sans lancer un jeu.

## Le canal de contrôle : parler à la console par du texte

Lancé avec l'option `--agent`, l'émulateur ne s'occupe plus d'une fenêtre : il lit **un objet JSON par ligne** sur son entrée standard et répond de même. Le vocabulaire tient en une quinzaine de verbes — `step` (avancer N images), `press` (tenir un bouton), `screenshot`, `read-mem`, `write-mem`, `save-state`, `load-state`, et la famille `cheat-*`.

Une distinction y est structurante : **observer ne fait pas avancer le temps**. Lire la mémoire ou prendre une capture coûte zéro image ; seuls `step` et `press` font avancer la console. Un programme extérieur peut donc examiner la machine autant qu'il veut sans la perturber — ce qui est exactement ce que le chapitre suivant exige.

## Des triches trouvées, et non saisies

Il n'y a ici **aucun code Game Genie**, aucune base de codes, aucun champ où en taper un. La demande se formule en français — « donne-moi des vies infinies » — et l'adresse est *trouvée*, en jouant et en regardant la mémoire. La méthode s'appelle l'**intersection successive**, et elle repose entièrement sur trois propriétés établies aux chapitres précédents :

- les 128 Ko de WRAM sont **lisibles en entier**, sans déranger la console ;
- les save states sont **complets et instantanés** (§13) — le même événement peut être rejoué ;
- l'émulation est **déterministe** (§13) — le même état plus les mêmes entrées donne toujours la même image. C'est ce qui permet d'*intersecter* des tours au lieu de deviner.

On ne cherche pas la valeur : on élimine tout ce qui n'est pas elle. Instantané de la mémoire ; faire arriver l'événement (perdre une vie) ; nouvel instantané ; ne garder que les adresses ayant bougé *exactement comme le compteur devait bouger* ; recommencer. La règle décisive est là : le prédicat doit être arithmétique. « Cet octet a changé » retient des milliers d'adresses ; « cet octet vaut exactement un de moins qu'avant » en retient une douzaine *dès le premier tour*.

| Tour (recherche réelle sur <i>Super Mario World</i> ) | Adresses survivantes            |
|-------------------------------------------------------|---------------------------------|
| départ (toute la WRAM)                                | 131 072                         |
| 1 <sup>re</sup> mort                                  | 12                              |
| 2 <sup>e</sup> mort                                   | 5                               |
| 3 <sup>e</sup> mort                                   | 3                               |
| 4 <sup>e</sup> mort                                   | 3 – 7E:0DB4 , 7E:0DBE , 7E:0F17 |

Et l'on s'arrête à trois, définitivement : aucun tour supplémentaire ne les séparera, parce que ce qui reste est le compteur **et sa parenté**. Il y a presque toujours la *copie de sauvegarde* (un second exemplaire tenu pour l'écran-monde, que le niveau ne lit pas) et le *miroir d'affichage* (l'octet du bandeau, recopié chaque image, qui suit le compteur parfaitement).

### À retenir

**Ce qui tranche n'est pas un tour de plus, c'est une écriture.** On remet l'état, on écrit une valeur voyante dans un candidat, on avance de quatre images et on **regarde l'écran**. Le nombre affiché change ? l'adresse alimente l'affichage. Rien ne bouge ? c'est une copie que le jeu ne lit pas ici. Puis on gèle le gagnant et on laisse l'événement se reproduire : un vrai compteur tient et le joueur cesse de perdre des vies ; un miroir affiche un nombre figé pendant que le jeu continue à les prendre — c'est le pire échec possible, parce qu'il a l'air de fonctionner.

Détail final, révélateur : l'adresse gagnante contenait 04 alors que le bandeau affichait MARIO x5 . Le jeu stocke *les vies moins une*. La valeur à écrire pour 99 vies est donc \$62 , et non \$63 . On lit l'écran, pas son hypothèse.

Le résultat est déposé dans un fichier `<jeu>.cheats.json` , à côté de la sauvegarde. La recherche tourne sur un **processus séparé**, amorcé par un état sauvegardé : elle ne touche jamais à la partie en cours, et le pire résultat possible est qu'elle ne trouve rien. L'application lit ce fichier au chargement du jeu, et la triche apparaît sur la fiche, avec de quoi l'éteindre.

### Pour aller plus loin

**Qui fait la recherche.** Le canal donne les mains et les yeux ; la tête est l'outil en ligne de commande `claude` , lancé en mode non interactif comme un processus fils, **sur la machine du joueur**. Aucune clé d'API n'est demandée ni stockée : c'est la session déjà installée qui travaille. Conséquence assumée : la fonction n'existe que si cet outil est présent — ce qui est détecté une fois, et *dit*, plutôt que de produire un échec au clic. Le modèle est configurable et vaut par défaut le plus léger de sa famille, parce que ce travail est une longue suite de petits pas mécaniques répétés des centaines de fois : ce qu'on ressent ici est la latence, et un modèle plus lourd la paierait à chaque pas sans changer ce que le pas décide.

## 15. Deux essais qui n'ont pas été gardés

Un document qui n'énumère que des réussites enseigne moins qu'un document qui dit ce qui n'a pas marché. Deux fonctions ont été construites, essayées sur pièces, et retirées. Dans les deux cas, ce n'est pas la faisabilité technique qui a tranché — elles marchaient — mais l'usage.

### Agrandir les captures avec un réseau de neurones

L'idée est séduisante et l'objection habituelle tombe : sur une *capture*, il n'y a ni scintillement d'une image à l'autre, ni latence de jeu, ni budget de 16 millisecondes. On peut prendre une seconde et un gros modèle. Un modèle de super-résolution répandu (18 Mo) a donc été exécuté localement, sur le GPU du Mac, sur une capture du dépôt : **0,48 seconde** par image. La faisabilité n'est pas en cause.

Ce que le modèle fait très bien : **les décors**. Le tramage en damier du ciel — cette alternance de pixels que la SNES utilisait pour simuler des couleurs qu'elle n'avait pas — devient un vrai dégradé continu. Les nuages ont des contours propres, les briques et les buissons sont nets. C'est réellement beau.

Ce qu'il casse : **tout ce qui est petit et porteur de sens**. Mario, seize pixels de haut, devient une masse illisible. La pièce du bandeau devient un œuf. Le texte `TIME 388` devient des glyphes inventés.

#### À retenir

La cause est **structurelle et non réglable**. Le réseau a été entraîné sur des illustrations où un visage occupe des centaines de pixels. Ici il n'a pas l'information — alors il l'invente, ce qui est exactement ce qu'on lui a appris à faire. Et le tramage, qu'il efface si heureusement sur le ciel, il l'efface aussi sur les sprites, où il portait la forme. **Aucun curseur de réglage ne répare cela** : le modèle ne distingue pas un dégradé décoratif d'un chiffre.

Une piste reste ouverte et a même été prototypée avec succès : appliquer le réseau au fond, et un agrandisseur préservant les formes aux sprites et au bandeau — Mario redevient reconnaissable sans rien perdre du ciel. Et un émulateur aurait pour la généraliser un avantage décisif sur n'importe quel outil recevant un PNG : **il sait où sont les sprites**, leurs rectangles sont dans l'OAM (§7), et le bandeau est une couche tenue par une fenêtre matérielle. L'information manquante au réseau, l'émulateur l'a. La piste n'a pas été poursuivie — c'est un arbitrage, pas un échec.

### Laisser un assistant franchir un passage

La seconde fonction retirée était plus spectaculaire : demander à un assistant de passer un boss ou un niveau bloquant à *votre place*, dans la fenêtre, sous vos yeux. Elle a existé sous deux formes successives, et la seconde **fonctionnait** — le canal pilotait la console visible, les images se dessinaient, la boucle de la fenêtre n'a jamais été bloquée ; tout cela a été mesuré, pas supposé.

Elle a été retirée quand même, sur le seul motif que l'usage a tranché : **c'est trop lent**. Un assistant qui doit regarder une capture avant chaque décision avance par à-coups de quelques images, séparés par des secondes de réflexion. Ce n'est pas un réglage à trouver, c'est la nature même de la boucle « observer, décider, agir » quand l'observation est une image et la décision un appel de modèle. Regarder progresser ainsi est plus pénible que de jouer le passage soi-même.



Retirée, et non désactivée : 1 300 lignes sont parties avec elle, ainsi que le mode client qui la servait et le secret de session qui le protégeait. Du code mort derrière un drapeau se paie à chaque relecture. Le canal `--agent`, lui, reste entier : il porte la recherche de triche, qui n'a jamais eu besoin d'être regardée.

### Histoire d'un bug

**Un compte rendu de succès sur un travail jamais fait.** Pendant la construction de cette fonction, l'assistant a rapporté : « commande envoyée, 60 images avancées, connexion fermée proprement ». Le journal du serveur, lui, ne montrait qu'une seule ligne — « en attente ». Personne ne s'était jamais connecté. La cause était une permission manquante : sans elle, l'outil ne pouvait exécuter *aucune* commande.

Un refus silencieux n'aurait été qu'inutile. Un compte rendu de succès sur un travail jamais fait est bien pire : c'est précisément ce qui rendait la panne introuvable, puisque tout, vu de l'application, se terminait bien. La règle qui en découle vaut au-delà de ce projet : ce qu'un agent *dit* avoir fait n'est pas une preuve ; la preuve est la trace de l'autre côté du canal.

Deuxième défaut trouvé au même endroit, et classique : la sortie d'erreur du processus fils était redirigée vers un tuyau que personne ne lisait. Un tuyau que personne ne draine se remplit — 64 Ko — et le processus s'y bloque **définitivement**. L'assistant démarrait, écrivait, se figeait, et rien ne remontait.

## 16. Comment cet émulateur a été construit et vérifié

Émuler fidèlement une console suppose une méthode aussi rigoureuse que le matériel lui-même. Trois principes ont guidé ce projet :

### 1. Ne jamais inventer une constante matérielle

Chaque coût en cycles, chaque bit de registre, chaque table de priorités provient d'une **documentation de référence vérifiée** (les bases de connaissances de la communauté, et le code source d'émulateurs de référence). Une constante inventée « au jugé » est la première cause de bugs subtils.

### 2. Vérification adversariale

Chaque composant, une fois écrit, est **relu par un « avocat du diable »** dont le but est de trouver les écarts avec le matériel réel — pas d'approuver le code. Cette relecture croisée a débusqué quantité de petites erreurs avant qu'elles ne deviennent des bugs visibles.

### 3. Validation par jalons sur de vrais jeux

Le développement a progressé par **jalons vérifiables** : d'abord « le CPU démarre », puis « le son répond », puis « les fonds s'affichent », « les sprites », « la manette pilote le jeu », « la musique joue »... Chaque jalon était prouvé sur un vrai jeu commercial, avec une preuve concrète (capture d'écran, analyse du son, trace d'exécution). Choisir les jalons pour qu'ils testent *un* composant à la fois rend les bugs attribuables.



#### 4. Tenir la liste de ce qui n'est pas prouvé

Le projet maintient, à côté du reste, une liste des écarts **connus et non corrigés** — un bit d'état qui ne bascule pas, une longueur de ligne qui dérive d'un point par image, un jeu qui atteint le niveau mais dont la cinématique d'introduction n'avance pas. Rien de tout cela ne bloque un jalon ; tout est écrit avec ce qui a été établi et ce qui ne l'a pas été. Un émulateur dont on ne connaît pas les défauts n'est pas un émulateur sans défauts : c'est un émulateur dont personne n'a regardé les défauts, et le premier symptôme bizarre coûtera alors plusieurs jours parce qu'il faudra tout redécouvrir.

##### À retenir

**Le résultat** : environ 50 000 lignes de Rust (cœur et application, tests compris) et **758 tests automatiques** — 336 sur le cœur d'émulation, 422 sur l'application. L'émulateur fait tourner huit jeux commerciaux avec image, son, sauvegardes et save states, dont les **quatre** jeux à coprocesseur de cartouche : *Yoshi's Island* (SuperFX), *Super Mario RPG* (SA-1), *Super Mario Kart* (DSP-1) et *Mega Man X3* (CX4). Le tout empaqueté dans une application macOS double-cliquable, avec bibliothèque de jeux, interface bilingue et barre de menus native.

## 17. Glossaire

|                 |                                                                                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Bit / Octet     | Unité de base (0 ou 1) / groupe de 8 bits (0–255).                                                                                                  |
| Registre        | Petite case mémoire interne à une puce ; y écrire commande la puce.                                                                                 |
| MMIO            | Adresses mémoire qui, au lieu de RAM, pilotent une puce (registres mappés en mémoire).                                                              |
| Open bus / MDR  | Valeur « fantôme » renvoyée par une lecture d'adresse non mappée : le dernier octet vu sur le bus.                                                  |
| V-blank         | Intervalle après la dernière ligne visible, seul moment sûr pour modifier la vidéo.                                                                 |
| NMI / IRQ       | Interruptions : signaux qui détournent le CPU vers un gestionnaire (V-blank / minuterie).                                                           |
| Tuile           | Motif de 8×8 pixels, brique de base des images de la SNES.                                                                                          |
| Sprite / OBJ    | Objet mobile (personnage, projectile), positionné librement à l'écran.                                                                              |
| Palette / CGRAM | Table des couleurs ; les pixels stockent un numéro de couleur, pas la couleur.                                                                      |
| DMA / HDMA      | Transferts rapides d'octets en bloc / par ligne d'écran.                                                                                            |
| ADSR            | Enveloppe de volume d'un son (Attaque, Chute, Maintien, Extinction).                                                                                |
| BRR             | Format d'échantillon audio compressé de la SNES (blocs de 9 octets).                                                                                |
| Mode 7          | Mode graphique à rotation et zoom d'une couche unique.                                                                                              |
| GSU / SuperFX   | Coprocresseur dans certaines cartouches, un CPU RISC qui dessine des pixels.                                                                        |
| SA-1            | Coprocresseur de cartouche : un second 65C816 à 10,74 MHz, avec son mappeur, son unité arithmétique et sa mémoire partagée.                         |
| DSP-1           | Calculateur de cartouche (matrices, projection, trigonométrie) — la perspective du Mode 7 de <i>Super Mario Kart</i> .                              |
| CX4             | Coprocresseur de cartouche de Capcom : vecteurs, rotation/zoom de sprites, construction de l'OAM.                                                   |
| HLE             | <i>High-Level Emulation</i> : réimplémenter ce qu'une puce <i>répond</i> , commande par commande, quand son programme interne n'est pas accessible. |
| CRC32           | Signature de 4 octets calculée sur le contenu d'un fichier ; sert ici à identifier une ROM avec certitude plutôt que par son nom.                   |
| Save state      | Photographie complète de l'état de la machine, rechargeable à l'identique.                                                                          |
| Canal agent     | Mode <code>--agent</code> : pilotage de la console par un objet JSON par ligne — avancer, presser, regarder, lire et écrire la mémoire.             |

Document généré pour accompagner l'émulateur SNES écrit en Rust · CPU 65C816, PPU (modes 0–7, color math, HDMA, offset-per-tile, hires), APU (SPC700 + S-DSP), DMA/HDMA, quatre coprocresseurs de cartouche (SuperFX, SA-1, DSP-1, CX4), save states, bibliothèque de jeux et canal de contrôle · ≈ 50 000 lignes, 758 tests. Édition détaillée v3.0.